Pontifícia Universidade Católica de Minas Gerais
Cursos: Engenharia de Software e Ciência de Dados e Inteligência Artificial
Disciplina: Teoria de Grafos e Computabilidade
Prof. Gabriel Fonseca **2026/2**

Trabalho Prático

Biblioteca de Grafos e Análise de Redes de Colaboração no GitHub

Orientações gerais

O trabalho poderá ser realizado em grupos de **até 5 alunos**. As implementações deverão ser desenvolvidas em **Java ou Python**, e o relatório final deverá ser produzido em LaTeX, utilizando o modelo oficial da Sociedade Brasileira de Computação (SBC).

O trabalho será dividido em **duas etapas**, com prazos de entrega definidos no Canvas. O código-fonte deverá ser versionado no repositório indicado pelo professor, inclusive no GitHub Classroom, quando aplicável. Todos os integrantes deverão contribuir efetivamente para o projeto, e o relatório final deverá explicitar as responsabilidades individuais.

É proibida a utilização de bibliotecas prontas de grafos, como NetworkX, JGraphT ou similares, para implementar as estruturas e os algoritmos exigidos. Bibliotecas auxiliares para requisições HTTP, tratamento de dados, testes e visualização são permitidas, desde que não substituam as implementações solicitadas. Cópias estarão sujeitas às penalidades acadêmicas aplicáveis.

1. Objetivo

Desenvolver uma ferramenta computacional para criação, manipulação e análise de grafos, aplicando conceitos de teoria dos grafos, orientação a objetos e engenharia de software. O projeto compreende:

1. **Etapa 1:** construção de uma biblioteca com API comum para grafos simples direcionados, implementada com matriz e lista de adjacência;
2. **Etapa 2:** coleta, modelagem e análise de interações entre colaboradores de um repositório público do GitHub, utilizando obrigatoriamente a biblioteca desenvolvida na Etapa 1.

2. Etapa 1 – Construção da Biblioteca de Manipulação de Grafos

Os grupos deverão desenvolver uma biblioteca para criação e manipulação de grafos simples e direcionados, com inserção e remoção de arestas, consultas de relações entre vértices, cálculo de graus, gerenciamento de pesos e exportação para visualização externa. A API deverá ser comum às diferentes representações internas.

2.1. Estrutura de classes

A biblioteca deverá conter obrigatoriamente:

- **Classe abstrata `AbstractGraph`:** define a API comum, atributos compartilhados (como número de vértices e pesos dos vértices), métodos auxiliares (como validação de índices) e métodos abstratos a serem implementados pelas classes concretas. Sempre que possível, comportamentos independentes da representação deverão ser compartilhados.

- **Classe concreta `AdjacencyMatrixGraph`:** implementa a API usando matriz de adjacência, inclusive para armazenar os pesos das arestas.
- **Classe concreta `AdjacencyListGraph`:** implementa a API usando listas de adjacência, dicionários, mapas ou estruturas equivalentes, incluindo os pesos das arestas.

As duas implementações deverão apresentar comportamento externo equivalente.

2.2. Construtores obrigatórios

- `AdjacencyMatrixGraph(int numVertices)`
- `AdjacencyListGraph(int numVertices)`

O número de vértices será fixado na criação do grafo. Os vértices terão índices inteiros de `0` até `numVertices - 1`.

2.3. API obrigatória

Método	Descrição
`int getVertexCount()`	Quantidade de vértices.
`int getEdgeCount()`	Quantidade de arestas.
`boolean hasEdge(int u, int v)`	Existência da aresta (u, v).
`void addEdge(int u, int v)`	Insere a aresta (u, v).
`void removeEdge(int u, int v)`	Remove a aresta (u, v).
`boolean isSuccessor(int u, int v)`	Verifica se v é sucessor de u.
`boolean isPredecessor(int u, int v)`	Verifica se u é predecessor de u.
`boolean isDivergent(int u1, int v1, int u2, int v2)`	Arestas existentes com mesma origem.
`boolean isConvergent(int u1, int v1, int u2, int v2)`	Arestas existentes com mesmo destino.
`boolean isIncident(int u, int v, int x)`	x é extremidade da aresta (u, v).
`int getVertexInDegree(int u)`	Grau de entrada de u.
`int getVertexOutDegree(int u)`	Grau de saída de u.
`void setVertexWeight(int v, double w)`	Define peso de vértice.
`double getVertexWeight(int v)`	Consulta peso de vértice.
`void setEdgeWeight(int u, int v, double w)`	Define peso de aresta existente.
`double getEdgeWeight(int u, int v)`	Consulta peso de aresta existente.
`boolean isConnected()`	Verifica conectividade fraca.
`boolean isEmptyGraph()`	Verifica ausência de arestas.
`boolean isCompleteGraph()`	Verifica completude direcionada.
`void exportToGEPHI(String path)`	Exporta em formato aceito pelo Gephi.

Em Python, as assinaturas poderão ser adaptadas quanto à anotação de tipos, preservando nomes, parâmetros e comportamentos.

2.4. Definições e restrições

- Os grafos serão **simples e direcionados**: não admitem laços nem múltiplas arestas. A adição de (u, u) é proibida.
- `addEdge(u,v)` será idempotente: chamadas repetidas não duplicam arestas.

- `isSuccessor(u,v)` e `isPredecessor(u,v)` retornam verdadeiro quando existe a aresta de u para v.
- Divergência e convergência serão verificadas para duas arestas existentes, respectivamente com mesma origem ou mesmo destino.
- `isConnected()` considerará **conectividade fraca**, desconsiderando a orientação das arestas.
- `isEmptyGraph()` retorna verdadeiro quando não existem arestas; `isCompleteGraph()` exige ambas as arestas (u, v) e (v, u) para todo par de vértices distintos.
- Pesos de vértices e arestas serão valores reais; os valores iniciais deverão ser definidos e documentados pelo grupo.
- Índices inválidos e operações inconsistentes, como consultar o peso de aresta inexistente, deverão lançar exceções.
- A biblioteca deverá manter consistentes contadores e atributos após as operações.
- `exportToGEPHI(path)` deverá gerar GEXF, GraphML ou CSV compatível com Gephi, preservando direções e pesos.

2.5. Programa de demonstração e testes

Deverá ser implementada uma aplicação separada, que demonstre as duas representações e todas as operações da API: criação de grafos, inserção e remoção de arestas, consultas de sucessores e predecessores, convergência, divergência e incidência, graus de entrada e saída, pesos, conectividade, grafo vazio, grafo completo e exportação para Gephi.

Os grupos deverão desenvolver **testes unitários** que verifiquem resultados esperados, casos-limite, tratamento de exceções e equivalência entre as representações.

Entregável da Etapa 1: código-fonte da biblioteca, programa de demonstração, testes unitários e documentação básica com instruções de execução e decisões de implementação.

3. Etapa 2 – Análise de Redes de Colaboração no GitHub

Nesta etapa, os grupos deverão utilizar a biblioteca da Etapa 1 para construir e analisar uma rede de colaboração entre usuários de um projeto de código aberto hospedado no GitHub. O objetivo é investigar interações, identificar colaboradores relevantes e compreender características estruturais da comunidade.

3.1. Seleção do repositório

Cada grupo deverá escolher um repositório público com comunidade ativa e volume significativo de interações. Recomenda-se um repositório com **mais de 5.000 estrelas**, histórico de *issues*, *pull requests* e múltiplos colaboradores. A escolha deverá ser justificada pela relevância do projeto, diversidade da comunidade, disponibilidade de dados e potencial de análise. O período de análise também deverá ser explicitado.

3.2. Coleta de dados

A aplicação deverá coletar informações públicas por meio da API REST ou GraphQL do GitHub, contemplando, no mínimo:

- comentários em *issues* e *pull requests*;

- abertura e fechamento de *issues*;
- abertura de *pull requests*;
- revisões e aprovações de *pull requests*;
- integração (*merge*) de *pull requests*.

A coleta deverá identificar os usuários envolvidos, o tipo e a direção das interações e informações necessárias à ponderação. Deverá tratar paginação, limites de requisições, erros e dados ausentes. Os dados deverão ser armazenados localmente para possibilitar a reprodução dos resultados sem nova coleta integral. Testes deverão validar a interpretação e transformação dos dados.

3.3. Modelagem dos grafos

Cada **vértice** representará um usuário do GitHub; cada **aresta direcionada** (u, v) representará uma interação de u em direção a v, segundo regras documentadas pelo grupo. Por exemplo, um comentário feito por u em uma *issue* aberta por v poderá produzir a aresta (u, v).

Deverão ser construídos **três grafos separados**:

1. **Rede de discussões:** comentários em *issues* e *pull requests*.
2. **Rede de gerenciamento de *issues*:** interações relacionadas à abertura e ao fechamento de *issues*, quando envolverem usuários distintos.
3. **Rede de revisão e integração de código:** revisões, aprovações e *merges* de *pull requests*.

Os grafos deverão usar identificadores de usuários consistentes para permitir sua integração.

Adicionalmente, deverá ser criado um **grafo consolidado**, combinando os diferentes tipos de interações. Como o grafo é simples, interações repetidas no mesmo par ordenado de usuários serão agregadas em uma única aresta ponderada.

Tipo de interação	**Peso sugerido**
Comentário em *issue* ou *pull request*	2
Fechamento de *issue* de outro usuário	3
Revisão ou aprovação de *pull request*	4
Merge de *pull request* de outro usuário	5

Os pesos deverão ser acumulados conforme a frequência das interações. Outras ponderações poderão ser utilizadas mediante justificativa. O grupo deverá evitar contagem dupla de eventos e documentar o tratamento de usuários automatizados, contas excluídas, interações consigo mesmo e eventos com múltiplos participantes.

3.4. Construção dos grafos com a biblioteca

A aplicação deverá identificar usuários e atribuir-lhes índices inteiros, manter a correspondência desses índices com as contas originais, instanciar os grafos utilizando as classes da Etapa 1, inserir e ponderar arestas por meio da API e exportar os resultados para o Gephi. **Não será permitido substituir a biblioteca própria por bibliotecas externas de grafos.**

3.5. Algoritmos da literatura

O grupo deverá **selecionar, estudar e implementar dois algoritmos ou métodos da literatura** aplicáveis à análise de redes de colaboração, fundamentados em artigos científicos, livros ou materiais acadêmicos reconhecidos. Os métodos deverão ir além das operações básicas da biblioteca.

Possibilidades incluem:

- **Centralidade e importância:** PageRank, HITS, centralidade de intermediação, centralidade de proximidade, centralidade de autovetor ou de Katz.
- **Estrutura e coesão:** coeficiente de agrupamento, assortatividade, análise de componentes e vulnerabilidade da rede.
- **Comunidades:** Girvan–Newman, propagação de rótulos e métodos baseados em modularidade.
- **Similaridade e difusão:** SimRank, modelos de propagação de influência e métodos espectrais.

Outros algoritmos poderão ser escolhidos, desde que adequados e fundamentados. Os **dois métodos deverão ser implementados pelo grupo**, usando a biblioteca da Etapa 1, sem recorrer a implementações prontas.

3.6. Aplicação e análise dos resultados

Os algoritmos deverão ser aplicados aos grafos construídos para responder a questões relevantes, por exemplo: quem são os colaboradores estruturalmente mais importantes? Quais usuários conectam grupos? Existem comunidades relativamente isoladas? Como os resultados variam entre as redes de discussões, gerenciamento e revisão? Como a ponderação altera os resultados?

Não é necessário responder a todas essas perguntas. A análise deverá formular questões coerentes com os métodos escolhidos e incluir tabelas, gráficos e visualizações dos grafos, preferencialmente no Gephi.

A discussão deverá abordar os resultados, sua interpretação no contexto do repositório, o impacto das decisões de modelagem, diferenças entre grafos específicos e consolidado, limitações dos dados e algoritmos e possíveis melhorias. Deve-se distinguir volume de participação de importância estrutural.

3.7. Aplicação de demonstração

A aplicação poderá ser executada em terminal e deverá permitir: carregar dados coletados, construir grafos, exibir número de vértices e arestas, executar os dois métodos selecionados, consultar resultados e exportar grafos. O procedimento deverá ser documentado para reprodução.

Entregável da Etapa 2: aplicação de coleta e análise, implementação dos dois algoritmos, dados ou instruções para sua obtenção, testes, arquivos de exportação e relatório final.

4. Documentação e Relatório Final

O relatório final deverá ser um artigo técnico-científico em LaTeX, utilizando o modelo oficial da SBC, com **7 a 15 páginas**. Deverá conter, no mínimo:

1. Introdução, contextualização do problema, objetivos e justificativa do repositório;
2. Fundamentação teórica dos conceitos de redes de colaboração e algoritmos escolhidos;
3. Descrição da biblioteca: arquitetura, classes, representações e decisões técnicas;
4. Coleta e modelagem: dados, interações, direção das arestas e pesos;
5. Metodologia de construção dos grafos e implementação dos algoritmos;
6. Resultados e discussão, incluindo visualizações e comparação das redes;

7. Limitações do trabalho;
8. Conclusão e trabalhos futuros;
9. Referências bibliográficas;
10. Identificação das contribuições individuais dos membros do grupo.

Modelo: Template de artigos da SBC no Overleaf.

5. Apresentação e Demonstração

Os grupos deverão apresentar o repositório escolhido e a modelagem, demonstrar a biblioteca e os algoritmos implementados, visualizar os grafos e discutir resultados e conclusões. Todos os integrantes deverão participar e estar preparados para responder sobre as diferentes partes do projeto. Duração e formato serão divulgados no Canvas.

6. Entregáveis

A entrega final deverá ser realizada no Canvas até as **23h59** da data indicada na atividade, em um único arquivo `.zip` ou `.rar` contendo:

1. Código-fonte completo da biblioteca de grafos;
2. Código-fonte da aplicação de coleta e análise do GitHub;
3. Implementações dos dois algoritmos selecionados;
4. Testes unitários;
5. Dados coletados ou scripts e instruções para obtê-los;
6. Grafos exportados para Gephi;
7. Relatório final em PDF e seus arquivos-fonte em LaTeX;
8. `README` com orientações de instalação, execução e reprodução.

O histórico de desenvolvimento no GitHub Classroom poderá ser considerado para avaliação das contribuições individuais.

7. Critérios de Avaliação

Critério	**Aspectos avaliados**
Biblioteca de grafos	Corretude da API, duas representações, abstração e reutilização.
Qualidade do software	Organização, exceções, testes, documentação e boas práticas.
Coleta e modelagem	Qualidade dos dados, definição das relações, consistência e construção dos grafos.
Algoritmos	Correção, fundamentação e adequação ao problema.
Análise dos resultados	Interpretações, visualizações e discussão crítica.
Relatório	Clareza, organização e qualidade técnico-científica.
Apresentação e participação	Domínio, demonstração e contribuição individual.

A distribuição de pontos e os prazos serão informados no Canvas. A solução deverá ser funcional e integrada: **a biblioteca própria é obrigatória também na Etapa 2**.